Code Explanation

Delta Live Tables (DLT) Pipeline Code Explanation

The code provided is a Delta Live Tables (DLT) pipeline written in Python, utilizing PySpark for data processing. It creates a series of tables by reading data from CSV files, cleaning and transforming the data, and then applying Slowly Changing Dimension (SCD) Type 2 logic to create a historical record of customer and order data.

Overview of the Code

This DLT pipeline consists of four main tables: `customer`, `order`, `ordersummary`, and `customeraggregatespend`. The pipeline reads data from CSV files, cleans and transforms the data, and applies SCD Type 2 logic to create a historical record of customer and order data. The final table provides aggregated customer spending data.

Step 1: Importing Necessary Modules and Defining Constants

This step involves importing the required PySpark modules and defining constants for data paths, catalog, and schema.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from delta.tables import DeltaTable
import dlt

# Define source data paths
CUSTOMER_DATA_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/
customerdata"
ORDER_DATA_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/ord
erdata"

# Define catalog and schema
CATALOG = "gen_ai_poc_databrickscoe"
SCHEMA = "sdlc_wizard"
```

Step 2: Creating the Customer Table

This step involves reading customer data from a CSV file, cleaning the data by removing duplicates and null values, and creating a `customer` table.

```
@dlt.table(
    name="customer",
    comment="Customer data cleaned from source"
)
def customer():
    customer_df = spark.read.format("csv").option("header", "true").
option("inferSchema", "true").load(CUSTOMER_DATA_PATH)
    cleaned_df = (customer_df
                  .dropDuplicates(["CustId"])
                  .filter(F.col("CustId").isNotNull() &
                          F.col("Name").isNotNull() &
                          F.col("EmailId").isNotNull() &
```

```
                        F.col("Region").isNotNull()))
    return cleaned_df
```

Step 3: Creating the Order Table

This step involves reading order data from a CSV file, calculating the `TotalAmount` column, cleaning the data by removing duplicates and null values, and creating an `order` table.

```
@dlt.table(
    name="order",
    comment="Order data cleaned from source with TotalAmount calcula
ted"
)
def order():
    order_df = spark.read.format("csv").option("header", "true").opt
ion("inferSchema", "true").load(ORDER_DATA_PATH)
    cleaned_df = (order_df
                  .withColumn("TotalAmount", F.col("PricePerUnit") *
F.col("Qty"))
                  .dropDuplicates(["OrderId"])
                  .filter(F.col("OrderId").isNotNull() &
                          F.col("ItemName").isNotNull() &
                          F.col("PricePerUnit").isNotNull() &
                          F.col("Qty").isNotNull() &
                          F.col("Date").isNotNull() &
                          F.col("CustId").isNotNull()))
    return cleaned_df
```

Step 4: Creating the Order Summary Table with SCD Type 2 Logic

This step involves joining the `customer` and `order` tables, applying SCD Type 2 logic to create a historical record of customer and order data, and creating an `ordersummary` table.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table joining customer and order data"
)
def ordersummary():
    customer_df = dlt.read("customer")
    order_df = dlt.read("order")
    # ... rest of the code for SCD Type 2 logic
```

The SCD Type 2 logic involves checking if the `ordersummary` table exists, and if not, creating it with the required columns. If the table exists, it updates the existing records by making them inactive and creates new records for changed customers.

```
if not table_exists:
    result_df = (joined_df
                 .withColumn("IsActive", F.lit(True))
                 .withColumn("StartDate", F.current_timestamp())
```

```
                .withColumn("EndDate", F.lit(None).cast("timestamp")
))
    return result_df
else:
    # ... rest of the code for updating existing records and creatin
g new records
```

Step 5: Creating the Customer Aggregate Spend Table

This step involves reading the `ordersummary` table, aggregating the `TotalAmount` column by `Name` and `Date`, and creating a `customeraggregatespend` table.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending data"
)
def customeraggregatespend():
    ordersummary_df = dlt.read("ordersummary")
    aggregated_df = (ordersummary_df
                     .filter(F.col("IsActive") == True)
                     .groupBy("Name", "Date")
                     .agg(F.sum("TotalAmount").alias("TotalAmount")))
    return aggregated_df
```